

Code-Layout, Readability and Reusability

Date	05 July 2026
Team ID	
Project Name	Smart Lender
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Standard PEP 8 formatting rules are strictly applied across all Python modules.
2	Proper File Structure	Files and folders are logically organized	Yes	Project split cleanly into static/ styling, templates/ views, and root controller files.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variable names match dataset fields and database models (e.g., applicant_income, loan_term).
4	Function / Method Names	Functions are descriptively named	Yes	Self-explanatory naming used for routes and database routines (e.g., load_prediction_model()).
5	Code Comments	Inline and block comments explain logic	Yes	Docstrings and comments explain scaling parameters, database schema links, and ML steps.
6	Modular Design	Code is split into reusable functions/modules	Yes	Model training, database design, and server routes are separated into distinct modules.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Removed all redundant data-mapping functions; shared functions are placed globally.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Handled with robust try-except blocks, SQLAlchemy database rollbacks, and UI flash alerts.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	load_prediction_model()	Python / Pickle	Main server setup (app.py) and API prediction controllers.	High

2	Database Schema models	Python / SQLAlchemy	Decoupled database initialization and route processing methods.	High
3	Base Layout Base template	HTML5 / Jinja2	Used as the parent template for home.html, predict.html, submit.html, and history.html.	High
4	Features Scaler Pipeline	Python / Scikit-learn	Reused across initial training scripts and real-time form input scaling.	High
5	Multi-step form wizard	JS / HTML	Reused on the prediction inputs screen to handle step validation.	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Follows modular MVC-style project structuring with clean, isolated layout folders.
Readability	5	Variables are descriptively named and code is structured with standardized indentation.
Reusability	4	Preprocessing scripts and HTML bases are modular and easy to adapt for other models.
Documentation / Comments	5	Code files are documented with comments, and a comprehensive README setup guide is included.
Overall Score	4.8/5	Represents a highly maintainable, structured, and production-ready implementation.